Chapter 2

Where Clause

1) SELECT employee_id, last_name, job_id, department_id
FROM employees
WHERE department_id = 90 ;
WHERE last_name = 'Whalen' ;
** formate DD-MON-RR.

Comparison Conditions

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
BETWEEN ...AND...	Between two values (inclusive)
IN(set)	Match any of a list of values
LIKE	Match a character pattern
IS NULL	Is a null value

!=and ^= also represent the not equal to

```
 SELECT last_name, salary FROM employees WHERE
salary <= 3000 ;
SELECT last_name, salary FROM employees WHERE
salary BETWEEN 2500 AND 3500 ;;
```

WHERE hire_date = '01-JAN-95' … WHERE salary >= 6000 … WHERE last_name = 'Smith'

*** WHERE last_name BETWEEN 'King' AND 'Smith';

**IN:

WHERE manager_id IN (100, 101, 201) ;

```
 WHERE salary BETWEEN 2500 AND 3500 ;
last_name BETWEEN 'King' AND 'Smith';
```

IN Condition:

```
  Select employee_id , last_name , salary ,
manager_id
from employees
where manager_id IN(100 , 101 , 200) ;
 WHERE last_name IN ('Hartstein', 'Vargas');
```

Like :

```
SELECT last_name FROM employees WHERE last_name
LIKE '_o%' ;  arekta “LIKE 'S%' ;  “
```

Escape option:

```
SELECT employee_id, last_name, job_id FROM
employees WHERE job_id LIKE '%SA\_%' ESCAPE '\';
```

WHERE commission_pct IS NULL ;

Logical Operators

```
   Select Job_id , employee_id, last_name ,
salary
from employees
where salary>=1000
And /OR   job_id like '%MAN%'  ;
```

```
SELECT last_name, job_id FROM employees WHERE
job_id NOT IN ('IT_PROG', 'ST_CLERK', 'SA_REP')
;
```

Precedence:

Ques: Select the row if an employee is a president and earns more than $15,000, or if the employee is a sales representative.

Ans :

```
 SELECT last_name, job_id, salary
FROM employees
WHERE job_id = 'SA_REP' OR job_id = 'AD_PRES'
AND salary > 15000;
```

2) Select the row if an employee is a president ora sales representative, and if the employee earns more than $15,000.

```
 SELECT last_name, job_id, salary
FROM employees
WHERE (job_id = 'SA_REP' OR job_id = 'AD_PRES')
AND salary > 15000;
```

ORDER by

**ORDER BY clause to display the rows in a specific order

```
1) SELECT employee_id, last_name, salary*12
annsal
```

```
FROM   employees
ORDER BY annsal ; [annsal  orderly]
2) SELECT last_name, department_id, salary FROM
employees ORDER BY department_id, salary DESC;
or ASC ;
```

Chapter 3

Case-Manipulation Functions

These functions convert case for character strings:

Function	Result
LOWER('SQL Course')	sql course
UPPER('SQL Course')	SQL COURSE
INITCAP('SQL Course')	Sql Course

lowercase

```
SELECT 'The job id for '||UPPER(last_name)||' is '
      ||LOWER(job_id) AS "EMPLOYEE DETAILS"
FROM   employees;
```

EMPLOYEE DETAILS
The job id for KING is ad_pres
The job id for KOCHHAR is ad_vp
The job id for DE HAAN is ad_vp
...
The job id for HIGGINS is ac_mgr
The job id for GIETZ is ac_account

Character-Manipulation Functions

These functions manipulate character strings:

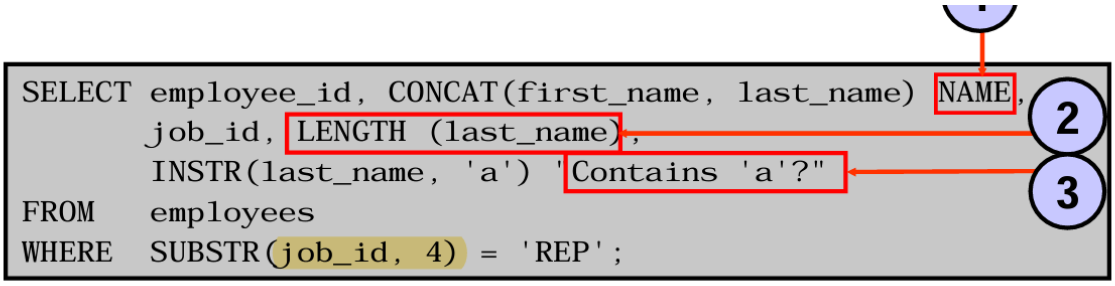
Function	Result
CONCAT('Hello', 'World')	HelloWorld
SUBSTR('HelloWorld',1,5)	Hello
LENGTH('HelloWorld')	10
INSTR('HelloWorld', 'W')	6
LPAD(salary,10,'*')	*****24000
RPAD(salary, 10, '*')	24000*****
REPLACE ('JACK and JUE','J','BL')	BLACK and BLUE
TRIM('H' FROM 'HelloWorld')	elloWorld

Character-Manipulation Functions

CONCAT, SUBSTR, LENGTH, INSTR, LPAD, RPAD, and TRIM are the character-manipulation functions that are covered in this lesson.

- CONCAT: Joins values together (You are limited to using two parameters with CONCAT.)
- SUBSTR: Extracts a string of determined length
- LENGTH: Shows the length of a string as a numeric value
- INSTR: Finds the numeric position of a named character
- LPAD: Pads the character value right-justified
- RPAD: Pads the character value left-justified
- TRIM: Trims heading or trailing characters (or both) from a character string (If *trim_character* or *trim_source* is a character literal, you must enclose it in single quotation marks.)

Note: You can use functions such as UPPER and LOWER with ampersand substitution. For example, use UPPER('&job_title') so that the user does not have to enter the job title in a specific case.



EMPLOYEE_ID	NAME	JOB_ID	LENGTH(LAST_NAME)	Contains 'a'?
174	EllenAbel	SA_REP	4	0
176	JonathonTaylor	SA_REP	6	2
178	KimberelyGrant	SA_REP	5	3
202	PatFay	MK_REP	3	2

Modify the SQL statement in the slide to display the data for those employees whose last names end with the letter n.

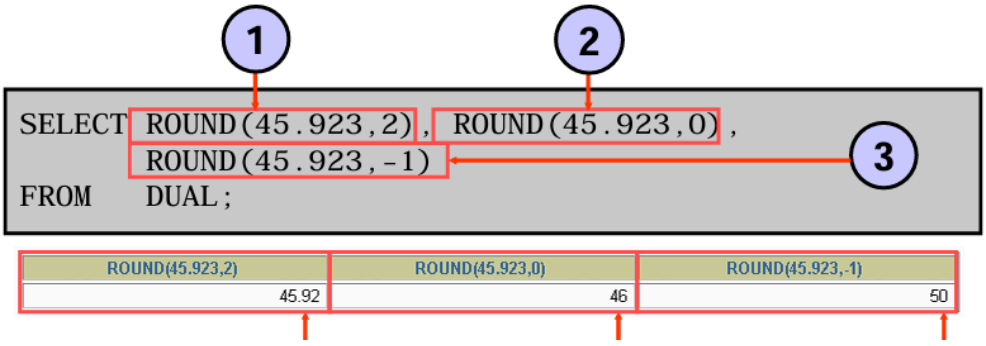
SELECT employee_id, CONCAT(first_name, last_name) NAME, LENGTH (last_name), INSTR(last_name, 'a') "Contains 'a'?" FROM employees WHERE SUBSTR(last_name, -1, 1) = 'n';

Number Functions

- ROUND: **Rounds value to specified decimal**
- TRUNC: **Truncates value to specified decimal**
- MOD: **Returns remainder of division**

Function	Result
ROUND(45.926, 2)	45.93
TRUNC(45.926, 2)	45.92
MOD(1600, 300)	100

Using the ROUND Function



SELECT last_name, salary, MOD(salary, 5000) FROM employees WHERE job_id = 'SA_REP';

Working with Dates :

1) SELECT last_name, hire_date FROM employees WHERE hire_date < '01-FEB-88';

Arithmetic with Dates

SELECT last_name, (SYSDATE-hire_date)/7 AS WEEKS FROM employees WHERE department_id = 90; [akahne SYSDATE is current time]

date + number	Date	Adds a number of days to a date
date – number	Date	Subtracts a number of days from a date
date – date	Number of days	Subtracts one date from another

Date Functions

Function	Result
MONTHS_BETWEEN	Number of months between two dates
ADD_MONTHS	Add calendar months to date
NEXT_DAY	Next day of the date specified
LAST_DAY	Last day of the month
ROUND	Round date
TRUNC	Truncate date

Date Functions

Date functions operate on Oracle dates. All date functions return a value of DATE data type except MONTHS_BETWEEN, which returns a numeric value.

- MONTHS_BETWEEN (*date1*, *date2*): Finds the number of months between *date1* and *date2*. The result can be positive or negative. If *date1* is later than *date2*, the result is positive; if *date1* is earlier than *date2*, the result is negative. The noninteger part of the result represents a portion of the month.
- ADD_MONTHS (*date*, *n*): Adds *n* number of calendar months to *date*. The value of *n* must be an integer and can be negative.
- NEXT_DAY (*date*, '*char*'): Finds the date of the next specified day of the week ('*char*') following *date*. The value of *char* may be a number representing a day or a character string.
- LAST_DAY (*date*): Finds the date of the last day of the month that contains *date*
- ROUND (*date*[,'*fmt*']): Returns *date* rounded to the unit that is specified by the format model *fmt*. If the format model *fmt* is omitted, *date* is rounded to the nearest day.
- TRUNC (*date*[, '*fmt*']): Returns *date* with the time portion of the day truncated to the unit that is specified by the format model *fmt*. If the format model *fmt* is omitted, *date* is truncated to the nearest day.

This list is a subset of the available date functions. The format models are covered later in this lesson. Examples of format models are month and year.

Using Date Functions

Function	Result
MONTHS_BETWEEN ('01-SEP-95','11-JAN-94')	19.6774194
ADD_MONTHS ('11-JAN-94',6)	'11-JUL-94'
NEXT_DAY ('01-SEP-95','FRIDAY')	'08-SEP-95'
LAST_DAY ('01-FEB-95')	'28-FEB-95'

SELECT employee_id, hire_date, MONTHS_BETWEEN (SYSDATE, hire_date) TENURE, ADD_MONTHS (hire_date, 6) REVIEW, NEXT_DAY (hire_date, 'FRIDAY'), LAST_DAY(hire_date) FROM employees WHERE MONTHS_BETWEEN (SYSDATE, hire_date) < 70;

Example : Compare the hire dates for all employees who started in 1997. Display the employee number, hire date, and start month using the ROUNDand TRUNCfunctions.

SELECT employee_id, hire_date, ROUND(hire_date, 'MONTH'), TRUNC(hire_date, 'MONTH') FROM employees WHERE hire_date LIKE '%97';

Insertion

**** Add a new tuple to *course***

insert into *course*

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

** or equivalently

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

** Add a new tuple to *student* with *tot_creds* set to null

insert into *student*
values ('3003', 'Green', 'Finance', *null*);

** Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of $18,000.

insert into *instructor*
select *ID, name, dept_name, 18000*
from *student*
where *dept_name* = Music' and *total_cred* > 144;

** The select from where statement is evaluated fully before any of its results are inserted into the relation.

Otherwise queries like

insert into *table*1 select * from *table*1

would cause problem